

LA32R 决赛汇编指令速查表

1. 整数加减

指令	汇编形式	功能 / 等价表达式	说明	示例
ADD.W	add.w rd, rj, rk	$rd = rj + rk$	32 位整数加法，结果保留低 32 位	add.w \$r6, \$r4, \$r5
SUB.W	sub.w rd, rj, rk	$rd = rj - rk$	32 位整数减法	sub.w \$r6, \$r4, \$r5
ADDI.W	addi.w rd, rj, si12	$rd = rj + sext(si12)$	12 位立即数先符号扩展；范围 $-2048 \sim 2047$ 。 非常适合指针递增和生成小常数	addi.w \$r4, \$r4, 4

常用写法

```
# r4 = 100
addi.w $r4, $r0, 100

# 指针向后移动一个 int
addi.w $r5, $r5, 4

# r6 = r4 + r5
add.w $r6, $r4, $r5
```

2. 整数比较

指令	汇编形式	功能 / 等价表达式	说明	示例
SLT	slt rd, rj, rk	$rd = ((signed)rj < (signed)rk) ? 1 : 0$	有符号比较，结果只会是 0 或 1	slt \$r6, \$r4, \$
SLTU	sltu rd, rj, rk	$rd = ((unsigned)rj < (unsigned)rk) ? 1 : 0$	无符号比较	sltu \$r6, \$r4,
SLTI	slti rd, rj, si12	$rd = ((signed)rj < sext(si12)) ? 1 : 0$	与 12 位有符号立即数进行有符号比较	slti \$r6, \$r4,
SLTUI	sltui rd, rj, si12	$rd = ((unsigned)rj < (unsigned)sext(si12)) ? 1 : 0$	立即数先符号扩展，然后按无符号数比较	sltui \$r6, \$r4,

注意

```
SLT / SLTI    → 有符号
SLTU / SLTUI  → 无符号
```

比较指令本身不跳转，只是生成 0/1 。

3. 逻辑运算

指令	汇编形式	功能 / 等价表达式	说明	示例
AND	and rd, rj, rk	$rd = rj \& rk$	按位与，常用于掩码操作	and \$r6, \$r4, \$r5
OR	or rd, rj, rk	$rd = rj rk$	按位或，常用于设置某些位	or \$r6, \$r4, \$r5
XOR	xor rd, rj, rk	$rd = rj \wedge rk$	按位异或，常用于翻转位或比较差异	xor \$r6, \$r4, \$r5

指令	汇编形式	功能 / 等价表达式	说明	示例
NOR	nor rd, rj, rk	$rd = \sim(rj \mid rk)$	按位 NOR；与 <code>\$r0</code> 配合可以实现 NOT	nor \$r6, \$r4, \$r0
ANDI	andi rd, rj, ui12	$rd = rj \& \text{zext}(ui12)$	12 位立即数零扩展，范围 <code>0 ~ 4095</code>	andi \$r5, \$r4, 0xff
ORI	ori rd, rj, ui12	$rd = rj \mid \text{zext}(ui12)$	12 位立即数零扩展，常用于置位	ori \$r5, \$r4, 1
XORI	xori rd, rj, ui12	$rd = rj \wedge \text{zext}(ui12)$	12 位立即数零扩展，常用于翻转位	xori \$r5, \$r4, 1

常用技巧

```
# 取最低 8 位
andi    $r5, $r4, 0xff
```

```
# 设置最低位
ori     $r5, $r4, 1
```

```
# 翻转最低位
xori    $r5, $r4, 1
```

```
# r5 = ~r4
nor     $r5, $r4, $r0
```

4. 移位运算

指令	汇编形式	功能 / 等价表达式	说明	示例
SLL.W	sll.w rd, rj, rk	$rd = rj \ll rk[4:0]$	逻辑左移，移位量取 <code>rk[4:0]</code>	sll.w \$r6, \$r4, \$r5
SRL.W	srl.w rd, rj, rk	$rd = rj \gg rk[4:0]$	逻辑右移，左侧补 0	srl.w \$r6, \$r4, \$r5
SRA.W	sra.w rd, rj, rk	$rd = \text{signed}(rj) \ggg rk[4:0]$	算术右移，左侧复制符号位	sra.w \$r6, \$r4, \$r5
SLLI.W	slli.w rd, rj, ui5	$rd = rj \ll ui5$	立即数逻辑左移， <code>ui5 = 0 ~ 31</code>	slli.w \$r5, \$r4, 2
SRLI.W	srli.w rd, rj, ui5	$rd = rj \gg ui5$	立即数逻辑右移，左侧补 0	srli.w \$r5, \$r4, 2
SRAI.W	srai.w rd, rj, ui5	$rd = \text{signed}(rj) \ggg ui5$	立即数算术右移，保留符号	srai.w \$r5, \$r4, 2

决赛中非常常用

```
# x *= 2
slli.w  $r4, $r4, 1
```

```
# x *= 4
slli.w  $r4, $r4, 2
```

```
# x *= 8
slli.w  $r4, $r4, 3
```

访问 32 位整数数组

```
# offset = i * 4
slli.w  $r6, $r5, 2
```

```
# addr = base + offset
add.w   $r6, $r4, $r6
```

对数组寻址而言，优先使用移位而不是 `MUL.W`。

5. 大常数与地址生成

指令	汇编形式	功能 / 等价表达式	说明	示例
LU12I.W	lu12i.w rd, si20	rd = si20 << 12	将 20 位立即数放入寄存器高 20 位，低 12 位清零；用于构造大常数和绝对地址	lu12i.w \$r4, 0x1c400
PCADDU12I	pcaddu12i rd, si20	rd = PC + (si20 << 12)	以当前 PC 为基准生成大范围 PC-relative 地址；可配合 Load/Store 或 JIRL 使用	pcaddu12i \$r4, 0x10

LU12I.W：构造大常数 / 绝对地址

例如：

```
# r4 = 0x1c400000
lu12i.w    $r4, 0x1c400
```

如果低 12 位不为 0，可以与 ORI 配合：

```
# r4 = 0x12345678
lu12i.w    $r4, 0x12345
ori        $r4, $r4, 0x678
```

可以记成：

```
高 20 位 → LU12I.W
低 12 位 → ORI
```

PCADDU12I：生成 PC 相对地址

```
pcaddu12i $r4, imm20
# r4 = 当前 PC + (imm20 << 12)
```

它与 LU12I.W 的区别：

```
LU12I.W    → 从 0 开始构造绝对数值 / 绝对地址
PCADDU12I  → 从当前 PC 开始构造相对地址
```

6. Load 访存指令

所有普通 Load 的有效地址：

```
address = rj + sext(si12)
```

指令	汇编形式	读取宽度	功能 / 等价表达式	说明	示例
LD.B	ld.b rd, rj, si12	8 bit	rd = sext(MEM8[rj + si12])	读取 1 字节并符号扩展到 32 位	ld.b \$r5, \$r4, 0
LD.BU	ld.bu rd, rj, si12	8 bit	rd = zext(MEM8[rj + si12])	读取 1 字节并零扩展	ld.bu \$r5, \$r4, 0
LD.H	ld.h rd, rj, si12	16 bit	rd = sext(MEM16[rj + si12])	读取半字并符号扩展	ld.h \$r5, \$r4, 0
LD.HU	ld.hu rd, rj, si12	16 bit	rd = zext(MEM16[rj + si12])	读取半字并零扩展	ld.hu \$r5, \$r4, 0
LD.W	ld.w rd, rj, si12	32 bit	rd = MEM32[rj + si12]	读取完整 32 位整数	ld.w \$r5, \$r4, 0

LD.B 与 LD.BU

若内存中的字节为：

```
0xFF
```

那么：

```
ld.b    $r5, $r4, 0
```

得到：

```
$r5 = 0xFFFFFFFF
```

而：

```
ld.bu   $r5, $r4, 0
```

得到：

```
$r5 = 0x000000FF
```

因此：

```
LD.B / LD.H    → 有符号数据
LD.BU / LD.HU  → 无符号数据
```

7. Store 访存指令

有效地址同样是：

```
address = rj + sext(si12)
```

指令	汇编形式	写入宽度	功能 / 等价表达式	说明	示例
ST.B	st.b rd, rj, si12	8 bit	MEM8[rj + si12] = rd[7:0]	只写 rd 最低 8 位	st.b \$r5, \$r4, 0
ST.H	st.h rd, rj, si12	16 bit	MEM16[rj + si12] = rd[15:0]	只写 rd 最低 16 位	st.h \$r5, \$r4, 0
ST.W	st.w rd, rj, si12	32 bit	MEM32[rj + si12] = rd	写完整 32 位	st.w \$r5, \$r4, 0

特别注意

```
st.w    $r5, $r4, 8
```

表示：

```
MEM32[$r4 + 8] = $r5
```

不是：

```
r5 = MEM[r4 + 8]
```

Store 指令中的：

rd = 数据
rj = 基地址
si12 = 地址偏移

8. 条件分支

指令	汇编形式	跳转条件	有符号性	说明	示例
BEQ	beq rj, rd, label	rj == rd	无关	相等则跳转	beq \$r4, \$r5, equal
BNE	bne rj, rd, label	rj != rd	无关	不相等则跳转	bne \$r4, \$r5, notequal
BLT	blt rj, rd, label	rj < rd	有符号	signed less than	blt \$r4, \$r5, loop
BGE	bge rj, rd, label	rj >= rd	有符号	signed greater/equal	bge \$r4, \$r5, end
BLTU	bltu rj, rd, label	rj < rd	无符号	unsigned less than	bltu \$r4, \$r5, loop
BGEU	bgeu rj, rd, label	rj >= rd	无符号	unsigned greater/equal	bgeu \$r4, \$r5, end

判断 0

```
# if (r4 == 0)
beq    $r4, $r0, is_zero

# if (r4 != 0)
bne    $r4, $r0, not_zero
```

普通循环

```
loop:
    # 循环体

    addi.w $r4, $r4, 1
    blt    $r4, $r5, loop
```

等价于:

```
do {
    ...
    r4++;
} while ((int32_t)r4 < (int32_t)r5);
```

9. 无条件跳转 / 函数调用

指令	汇编形式	功能 / 等价表达式	说明	示例
B	b label	PC = target	无条件 PC-relative 跳转	b loop
BL	bl label	\$r1 = PC + 4; PC = target	跳转同时把返回地址写入 \$r1 , 主要用于函数调用	bl function
JIRL	jirl rd, rj, offs16	rd = PC + 4; PC = rj + sext(offs16 << 2)	寄存器间接跳转, 可用于函数返回	jirl \$r0, \$r1, 0

函数调用

```
bl    function
```

相当于：

```
$r1 = 返回地址
PC  = function
```

函数返回

```
jirl    $r0, $r1, 0
```

因为写 `$r0` 会被丢弃，所以：

```
PC = $r1
```

也就是从函数返回。

10. 乘法

指令	汇编形式	功能 / 等价表达式	说明	示例
MUL.W	mul.w rd, rj, rk	rd = (rj * rk)[31:0]	32×32 位乘法，只保留乘积低 32 位；当前 CPU 不提供高 32 位乘法结果	mul.w \$r6, \$r4, \$r5

例如：

```
addi.w $r4, $r0, 10
addi.w $r5, $r0, 20
mul.w  $r6, $r4, $r5
```

结果：

```
$r6 = 200
```

11. 全部 40 条指令总表

这个版本最适合决赛现场快速搜索。

类别	指令	汇编形式	功能	说明
加减	ADD.W	add.w rd, rj, rk	rd = rj + rk	寄存器加法
加减	SUB.W	sub.w rd, rj, rk	rd = rj - rk	寄存器减法
加减	ADDI.W	addi.w rd, rj, si12	rd = rj + sext(si12)	小常数、指针递增
比较	SLT	slt rd, rj, rk	rd = signed(rj) < signed(rk)	有符号，小于返回 1
比较	SLTU	sltu rd, rj, rk	rd = unsigned(rj) < unsigned(rk)	无符号比较
比较	SLTI	slti rd, rj, si12	rd = signed(rj) < sext(si12)	有符号立即数比较
比较	SLTUI	sltui rd, rj, si12	无符号比较	立即数先符号扩展

类别	指令	汇编形式	功能	说明
逻辑	AND	and rd, rj, rk	$rd = rj \& rk$	按位与
逻辑	OR	or rd, rj, rk	$rd = rj rk$	按位或
逻辑	XOR	xor rd, rj, rk	$rd = rj \wedge rk$	按位异或
逻辑	NOR	nor rd, rj, rk	$rd = \sim(rj rk)$	nor rd,rj,\$r0 可实现 NOT
逻辑	ANDI	andi rd, rj, ui12	$rd = rj \& \text{zext}(ui12)$	常用于掩码
逻辑	ORI	ori rd, rj, ui12	$rd = rj \text{zext}(ui12)$	常用于置位
逻辑	XORI	xori rd, rj, ui12	$rd = rj \wedge \text{zext}(ui12)$	常用于翻转位
移位	SLL.W	sll.w rd, rj, rk	$rd = rj \ll rk[4:0]$	逻辑左移
移位	SRL.W	srl.w rd, rj, rk	$rd = rj \gg rk[4:0]$	逻辑右移, 补 0
移位	SRA.W	sra.w rd, rj, rk	$rd = \text{signed}(rj) \ggg rk[4:0]$	算术右移, 补符号位
移位	SLLI.W	slli.w rd, rj, ui5	$rd = rj \ll ui5$	乘 2^n 很实用
移位	SRLI.W	srli.w rd, rj, ui5	$rd = rj \gg ui5$	无符号右移
移位	SRAI.W	srai.w rd, rj, ui5	$rd = \text{signed}(rj) \ggg ui5$	有符号算术右移
常数	LU12I.W	lu12i.w rd, si20	$rd = si20 \ll 12$	构造大常数/绝对地址, 低 12 位清零
地址	PCADDU12I	pcaddu12i rd, si20	$rd = PC + (si20 \ll 12)$	构造 PC-relative 大地址
Load	LD.B	ld.b rd, rj, si12	读取 8 位并符号扩展	signed byte
Load	LD.BU	ld.bu rd, rj, si12	读取 8 位并零扩展	unsigned byte
Load	LD.H	ld.h rd, rj, si12	读取 16 位并符号扩展	signed halfword
Load	LD.HU	ld.hu rd, rj, si12	读取 16 位并零扩展	unsigned halfword
Load	LD.W	ld.w rd, rj, si12	读取 32 位	最常用数组读取
Store	ST.B	st.b rd, rj, si12	写入 rd[7:0]	写一个字节
Store	ST.H	st.h rd, rj, si12	写入 rd[15:0]	写半字
Store	ST.W	st.w rd, rj, si12	写入完整 rd	最常用数组写入
分支	BEQ	beq rj, rd, label	相等跳转	==
分支	BNE	bne rj, rd, label	不相等跳转	!=
分支	BLT	blt rj, rd, label	signed <	有符号
分支	BGE	bge rj, rd, label	signed >=	有符号
分支	BLTU	bltu rj, rd, label	unsigned <	无符号
分支	BGEU	bgeu rj, rd, label	unsigned >=	无符号
跳转	B	b label	无条件跳转	普通循环常用
跳转	BL	bl label	跳转并令 $\$r1 = PC + 4$	函数调用
跳转	JIRL	jirl rd, rj, ofs16	寄存器间接跳转	jirl \$r0,\$r1,0 返回
乘法	MUL.W	mul.w rd, rj, rk	$rd = \text{product}[31:0]$	只取低 32 位

12. 决赛最常用的 C → LA32R 对照

C / 目标操作	LA32R 写法	说明
x = y + z	add.w \$x, \$y, \$z	加法
x = y - z	sub.w \$x, \$y, \$z	减法
x++	addi.w \$x, \$x, 1	自增
x--	addi.w \$x, \$x, -1	自减
x = 100	addi.w \$x, \$r0, 100	小常数生成
x = 0x1c400000	lu12i.w \$x, 0x1c400	大常数/绝对地址
x = 0x12345678	lu12i.w \$x, 0x12345 + ori \$x, \$x, 0x678	任意 32 位常数
x = PC + 大偏移	pcaddu12i \$x, imm20	PC 相对地址生成
x = a * b	mul.w \$x, \$a, \$b	普通乘法
x = a * 2	slli.w \$x, \$a, 1	优先移位
x = a * 4	slli.w \$x, \$a, 2	数组地址常用
x = a * 8	slli.w \$x, \$a, 3	优先移位
x = a & 0xff	andi \$x, \$a, 0xff	取低 8 位
x = ~a	nor \$x, \$a, \$r0	NOT
x = A[i]	slli.w offset,i,2 + add.w addr,base,offset + ld.w	int 数组
A[i] = x	计算 base+i*4 后 st.w	int 数组写入
if (a == b)	beq \$a, \$b, label	相等
if (a != b)	bne \$a, \$b, label	不等
if ((int)a < (int)b)	blt \$a, \$b, label	signed
if ((uint)a < (uint)b)	bltu \$a, \$b, label	unsigned
if (a == 0)	beq \$a, \$r0, label	判零
if (a != 0)	bne \$a, \$r0, label	判非零
goto label	b label	无条件跳转
func()	bl func	函数调用
return	jirl \$r0, \$r1, 0	函数返回